Q1. RISC-V Registers
- List and name the 32 general-purpose registers in RISC-V.
-Detail their specific features and conventional usage (ABI names like zero, ra, sp, gp, tp, t0-t6, s0-s11, a0-a7).

Ans:

**RISC-V Registers** are **small, fast storage locations inside the CPU** used to hold data, addresses, an instructions during program execution.

## Simple definition (exam ready):

**RISC-V registers are CPU memory units that temporarily store operands, results, and addresses to perform operations efficiently.**

## Key points:

RISC-V has **32 general-purpose registers**
Named **x0 to x31**
All registers are **same size**
**32-bit** in RV32
**64-bit** in RV64

## General-Purpose Registers in RISC-V (32 Registers)

RISC-V provides **32 General-Purpose Registers (GPRs)** used to store data, addresses, and intermediate results.

## Key points

- Registers are named **x0 to x31**
- All are **same size**
- **32-bit** in RV32
- **64-bit** in RV64
- Used for **arithmetic, logic, data movement, and addressing**
- **x0 is hard-wired to 0**
-

| x Register | ABI Name | Purpose |
|---|---|---|
| x0 | zero | Constant 0 |
| x1 | ra | Return address |
| x2 | sp | Stack pointer |
| x3 | gp | Global pointer |
| x4 | tp | Thread pointer |
| x5–x7 | t0–t2 | Temporary |
| x8 | s0 / fp | Saved / Frame pointer |
| x9 | s1 | Saved |
| x10–x17 | a0–a7 | Arguments / Return values |
| x18–x27 | s2–s11 | Saved registers |

| x Register | ABI Name | Purpose |
|---|---|---|
| x28–x31 | t3–t6 | Temporary |

B,

**Special Purpose Register (SPR)**

**A** Special Purpose Register **is a CPU register designed for a** specific control or coordination task**, not for general data storage.**

**(a) Program Counter (PC)**

- Holds the **address of the next instruction** to be executed
- Automatically **updates after each instruction**
- Modified during **branch and jump** instructions

**(b) Instruction Register (IR)**

- Stores the **currently fetched instruction**
- The instruction is **decoded and executed** from IR
- Used **internally by the CPU** during execution

.....

## Question 2: RISC-V Architecture Design Principles

Explain the **four fundamental design principles of the RISC-V architecture**.
Illustrate each principle with **three suitable examples**.

The design principles are:

1. Regularity supports simplicity
2. Make the common case fast
3. Smaller is faster
4. Good design demands good compromises

# Ans:

Design Principles **are the** basic rules or ideas **followed while** designing the RISC-V processor**.**

RISC-V follows **four fundamental design principles** to achieve simplicity, speed, and efficiency.

### 1. Regularity Supports Simplicity

**Meaning** Regularity means using the same rules and formats everywhere.This makes the CPU hardware simpler, faster, and easier to design..

1. Fixed **32-bit instruction length**
2. **32 general-purpose registers** with uniform size

3. Same instruction format for arithmetic operations (e.g., `add`, `sub`)

## 2. Make the Common Case Fast

**Meaning:** Optimize operations that occur most frequently.

**Examples:**

1. Register-to-register operations are faster than memory access
2. Load/store architecture (only load & store access memory)
3. Frequently used instructions (add, load, branch) execute quickly

## 3. Smaller is Faster

**Meaning:** Simpler hardware improves speed.

**Examples:**

1. Simple instruction set (reduced complexity)
2. Few addressing modes
3. Small, fast register file improves access time

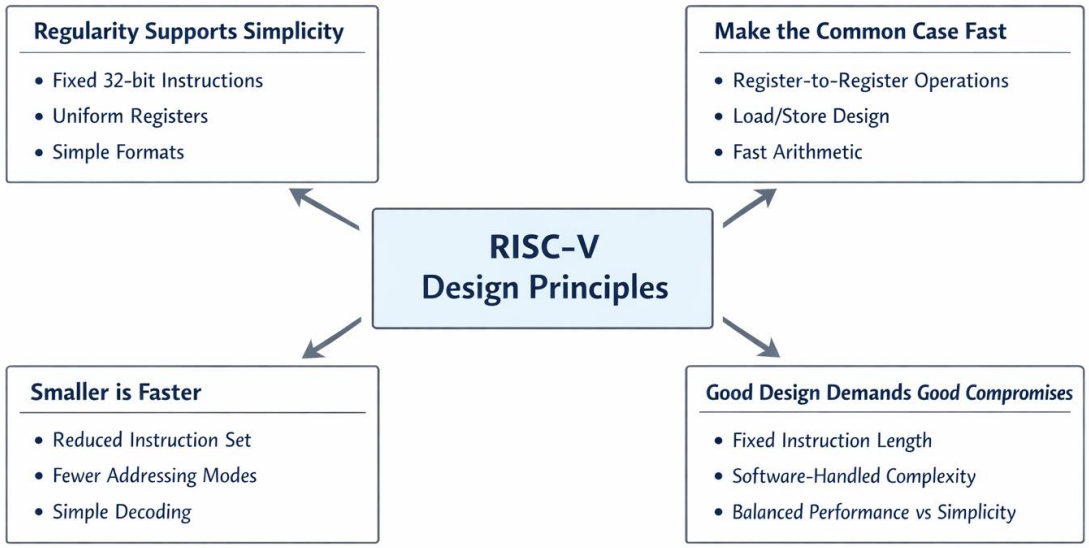
## 4. Good Design Demands Good Compromises

**Meaning:** Balance performance, cost, and simplicity.

**Examples:**

1. Fixed instruction length instead of variable length
2. Limited instruction formats for easier decoding
3. Software handles complex operations instead of hardware

**Conclusion (Exam Ending Line):**
RISC-V architecture uses simplicity, speed optimization, compact design, and balanced trade-offs to achieve efficient processor performance.




.....

**Q3. Architecture Comparison & Directives**

**a, Compare and contrast RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer) architectures.**

**b, What are Assembler Directives? Explain the various directives used specifically in RISC-V assembly language. Part B: Memory Organization & Data Representation**

**Ans :**

A, **RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer) architectures.**

| Feature | RISC | CISC |
|---|---|---|
| Instruction set | Small, simple | Large, complex |
| Instruction length | Fixed (usually 32-bit) | Variable |
| Execution | One instruction per cycle (mostly) | Multiple cycles per instruction |

| Feature | RISC | CISC |
|---|---|---|
| Memory access | Load/Store only | Instructions can access memory |
| Registers | Large register file | Fewer registers |
| Hardware design | Simple | Complex |
| Examples | **RISC-V, ARM** | **x86, Intel 8086** |

B, **Assembler Directives**

An **assembler** is a **system software** that **converts assembly language into machine language**.

Assembler directives tell the assembler **how to store data, where to place code, and how to organize a program**.
Assembler directives are **instructions to the assembler**, not the CPU.
They **do not generate machine code** but help in **program organization and data definition**.

## Common RISC-V Assembler Directives:

| Directive | Purpose |
|---|---|
| `.data` | Defines data section |
| `.text` | Defines code (instruction) section |
| `.globl label` | Makes a label global |
| `.word` | Allocates 32-bit data |
| `.byte` | Allocates 8-bit data |
| `.half` | Allocates 16-bit data |
| `.asciz` | Stores null-terminated string |
| `.align n` | Aligns data in memory |
| `.end` | Marks end of program |

**Q4. Byte Addressable Memory**

A, What do you mean by Byte Addressable Memory?

B, Explain the two different types of byte assignment (Big Endian and Little Endian) using neat diagrams.

**Ans :**

**A,**

Byte Addressable Memory means:

- Each memory address stores 1 byte (8 bits).
- Every byte has a unique address.
- Larger data (word, int) uses multiple consecutive addresses.

## Simple definition:

In byte addressable memory, each memory location stores one byte, and every byte has its own address.

## Example:

- A 32-bit integer (4 bytes) needs 4 memory addresses.

**B,**

Byte Assignment (Endianness):

Endianness is the rule that decides **which byte is stored first in memory** when data uses more than one byte.

Example: an **integer** like `1234` needs **4 bytes**.

There are two types:

1. Big Endian
2. Little Endian

### 1) Big Endian

Meaning: Most Significant Byte (MSB) stored at the lowest address.

Diagram (neat & exam-safe):

Assume 32-bit value: 0x12 34 56 78
Starting address: 0x1000

| Address | | Data | |
|---|---|---|---|
| 0x1000 | → | 12 | (MSB) |
| 0x1001 | → | 34 | |
| 0x1002 | → | 56 | |
| 0x1003 | → | 78 | (LSB) |

## 2) Little Endian

Meaning: Least Significant Byte (LSB) stored at the lowest address.

Diagram:

Same value: 0x12 34 56 78
Starting address: 0x1000

**Address Data**

0x1000 → 78 (LSB)

0x1001 → 56

0x1002 → 34

0x1003 → 12 (MSB)

| Feature | Big Endian | Little Endian |
|---|---|---|
| Lowest address | MSB | LSB |
| Human readability | Easy | Hard |
| RISC-V uses | ✗ | ✓ |

**Q5. Memory Mapping Exercise**

**Assume a 32-bit (4-byte) integer with the hexadecimal value 0x56327814 is stored in memory starting at address 0x1000. Write the Big Endian and Little Endian assignments for this specific value.**

**Ans :**

**Given:**

Data size: **32-bit integer = 4 bytes**

Hex value: **0x56327814**

Starting memory address: **0x1000**

Split the 32-bit value into bytes:

**0x56** 0x32 0x78 **0x14**

**MSB** **LSB**

## 1) Big Endian Assignment

 **Most Significant Byte (MSB)** is stored at the **lowest address**.

**Address Data (Hex)**
**0x1000 → 56(msb)**

**0x1001 → 32**
**0x1002 → 78**
**0x1003 → 14(lsb)**

## 2) Little Endian Assignment

 **Least Significant Byte (LSB)** is stored at the **lowest address**.

**Address Data (Hex)**
**0x1000 → 14(lsb)**
**0x1001 → 78**
**0x1002 → 32**
**0x1003 → 56(msb)**

**Q6. ASCII & String Storage**

A, Write the following strings using ASCII encoding in hexadecimal:

(a) hello there * (b) bag o chips * (c) To the rescue!

B,Show how these strings are stored in byte-addressable memory starting at address 0x004F05BC. Clearly indicate the memory address of each byte, assuming the first character is at the lowest address.

**Ans:**

Each character is stored as **1 byte (8 bits)** using its **ASCII hexadecimal code**.
In byte-addressable memory, **each byte gets one address**.
Hex:
61 a 62 b 63 c 64 d 65 e
66 f 67 g 68 h 69 i 6A j
6B k 6C l 6D m 6E n 6F o
70 p 71 q 72 r 73 s 74 t
75 u 76 v 77 w 78 x 79 y
7A z

| Character | Decimal | Hex |
|---|---|---|
| A | 65 | 41 |
| a | 97 | 61 |
| 0 | 48 | 30 |
| space | 32 | 20 |
| ! | 33 | 21 |

(a) “hello there”

ASCII (Hex):

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| h | e | l | l | o | | t | h | e | r | e |
| 68 | 65 | 6C | 6C | 6F | 20 | 74 | 68 | 65 | 72 | 65 |

Memory storage (starting at 0x004F05BC):

| Address | Data |
|---|---|
| 0x004F05BC | 68 |
| 0x004F05BD | 65 |
| 0x004F05BE | 6C |
| 0x004F05BF | 6C |
| 0x004F05C0 | 6F |
| 0x004F05C1 | 20 |
| 0x004F05C2 | 74 |
| 0x004F05C3 | 68 |
| 0x004F05C4 | 65 |
| 0x004F05C5 | 72 |
| 0x004F05C6 | 65 |

(b) “bag o chips”

ASCII (Hex):

B a g o c h i p s

62 61 67 20 6F 20 63 68 69 70 73

Memory storage:

| Address | Data |
|---|---|
| 0x004F05BC | 62 |
| 0x004F05BD | 61 |
| 0x004F05BE | 67 |
| 0x004F05BF | 20 |
| 0x004F05C0 | 6F |
| 0x004F05C1 | 20 |
| 0x004F05C2 | 63 |
| 0x004F05C3 | 68 |
| 0x004F05C4 | 69 |
| 0x004F05C5 | 70 |
| 0x004F05C6 | 73 |

(c) “To the rescue!”

ASCII (Hex):

T o t h e r e s c u e !

54 6F 20 74 68 65 20 72 65 73 63 75 65 21

Memory storage:

| Address | Data |
|---|---|
| 0x004F05BC | 54 |
| 0x004F05BD | 6F |
| 0x004F05BE | 20 |
| 0x004F05BF | 74 |
| 0x004F05C0 | 68 |
| 0x004F05C1 | 65 |
| 0x004F05C2 | 20 |
| 0x004F05C3 | 72 |
| 0x004F05C4 | 65 |
| 0x004F05C5 | 73 |
| 0x004F05C6 | 63 |
| 0x004F05C7 | 75 |
| 0x004F05C8 | 65 |
| 0x004F05C9 | 21 |

**……**

**Q7. Immediate Values**

A, Explain how to place immediates in registers using addi and lui.

B, Write RISC-V assembly code to place the following constants in register s7 using the minimum number of instructions:

(a) 29 (b) –214 (c) –2999 (d) 0xABCDE000 (e) 0xEDCBA123 (f) 0xEEEEEFAB

**Ans :**

## Immediate Values in RISC-V

An immediate is a constant value written directly inside an instruction

. RISC-V mainly uses **two instructions** to load immediates into registers:

```
addi , lui
```

## Using `addi` to put a number into a register

The instruction looks like this:

```
addi rd, rs1, imm
```

- `rd` → the register where the result goes
- `rs1` → the source register (the starting value)
- `imm` → the number you want to add

### Step 1: If you want to load a number directly

Registers can start from 0 using **x0**, which is always 0.

Example: Put **29** in `s7`:

```
addi s7, x0, 29
```

- `x0` = 0 → starting value
- `imm` = 29 → the number to add
- Result goes into `s7` → now `s7 = 29` ✅

### Step 2: Add a number to an existing register

Example: `x1 = 10`, now add 5:

```
addi x2, x1, 5
```

- `x1` = 10 → starting value
- `imm` = 5 → add this number
- Result goes into `x2` → now `x2 = 15` ✅

### Step 3: Add a negative number

Example: `x3 = 20`, subtract 7:

```
addi x4, x3, -7
```

- `x3` = 20
- `imm` = -7
- Result → `x4 = 13` ✅

## Load Upper Immediate (`lui`) – Exam POV

**Purpose:**

- Load a **20-bit immediate** into the **upper 20 bits** of a 32-bit register.
- The **lower 12 bits** are automatically set to **0**.

**Syntax:**

```
lui rd, imm20
```

**Steps to place a number in a register:**

1. **Decide the full 32-bit number you want**.
   Example: `0x12345678`
2. **Split it into upper 20 bits and lower 12 bits**:
- Upper 20 bits = `0x12345`
- Lower 12 bits = `0x678`
3. **Load upper bits using `lui`:**

```
lui x5, 0x12345    # x5 = 0x12345000
```

4. **Add lower bits using `addi`:**

```
addi x5, x5, 0x678  # x5 = 0x12345678
```

**B,**

### (a) 29

✔ Fits in 12-bit immediate → **1 instruction**

```
addi s7, x0, 29
```

### (b) −214

✔ Fits in 12-bit immediate → **1 instruction**

```
addi s7, x0, -214
```

### (c) −2999

✕ Too large for `addi` alone
✔ Use `lui + addi`

```
lui s7, -1
addi s7, s7, 1097
```

### (d) 0xABCDE000

✔ Lower 12 bits are zero → **only `lui` needed**

```
lui s7, 0xABCDE
```

### (e) 0xEDCBA123

✕f Lower 12 bits not zero → **2 instructions**

Split:

- Upper 20 bits = `0xEDCBA`
- Lower 12 bits = `0x123`

```
lui s7, 0xEDCBA
 addi s7, s7, 0x123
```

**(f) 0xEEEEEFAB**

Split:

- Upper 20 bits = `0xEEEEE`
- Lower 12 bits = `0xFAB` (negative signed value)

```
lui s7, 0xEEEEE
addi s7, s7, -85
```

……..

## Q8. Logical Operations (NAND/NOR Implementation)

**A**, Implement the NOR operation using OR and XORi. (Target: x5, Inputs: x6, x7).
**B** , Implement the NAND operation using AND and XORi. (Target: x5, Inputs: x6, x7).
**C,** Justify your answer: Explain why XORi with -1 is used to achieve the NOT/Inversion effect.

**Ans:**

### (a) Implement NOR using OR and XORi

NOR Definition:

NOR(A,B)=NOT(A OR B)

**RISC-V Trick**

- RISC-V does **not have a direct NOT instruction**
- Use **XORi with −1** to invert bits

x⊕0xFFFFFFFF=NOT(x)$x \oplus 0xFFFFFFFF$=NOT($x$)

Steps (x6 and x7 as inputs, x5 as output)

1. Perform OR:

```
or x5, x6, x7 # x5 = x6 | x7
```

2. Invert result using XORi with -1:

```
xori x5, x5, -1 # x5 = NOT(x5)
```

✅ Now x5 contains **NOR(x6, x7)**

**(b) Implement NAND using AND and XORi**

NAND Definition

NAND(A,B)=NOT(A AND B)

Steps:

1. Perform AND:

```
and x5, x6, x7 # x5 = x6 & x7
```

2. Invert result using XORi with -1:

```
xori x5, x5, -1 # x5 = NOT(x5)
```

✅ Now x5 contains **NAND(x6, x7)**

## (c) Justification

### Step 1: What is XOR?

XOR (⊕) is a rule for flipping bits:

- `0 ⊕ 0 = 0`
- `0 ⊕ 1 = 1` ✅flips
- `1 ⊕ 0 = 1`
- `1 ⊕ 1 = 0` ✅flips

✅**Key idea:** XOR with 1 **flips the bit**.

---

### Step 2: What is -1 in binary?

In computers, -1 is written as **all 1s**:

```
32-bit: 11111111 11111111 11111111 11111111
Hex:    0xFFFFFFFF
```

- Each bit is 1.

---

### Step 3: XOR with -1

If you do:

```
x ⊕ -1
```

- Every bit of `x` is XORed with 1 → **every bit flips**

**Example (8 bits for simplicity):**

```
x = 10101010
-1 = 11111111
x ⊕ -1 = 01010101  # every bit flipped
```

- That is exactly the **NOT(x)**

---

### Step 4: So in simple words

- XOR with -1 = flip all bits
- Flipping all bits = **NOT / inversion**

**……**

### Q9. Control Flow

Demonstrate how to convert High-Level Code (specifically If-Else statements) into RISC-V assembly code.

## Ans :

High-Level Code Example:

……

**if (a > b) {**
**c = a;**
**} else {**
**c = b;**
**}**

**……..**

Assume registers:

- `a` → x10 (`a0`)
- `b` → x11 (`a1`)
- `c` → x12 (`a2`)

### Step 1: Compare a and b

RISC-V uses **slt (set less than)** to set a flag:

```
slt x5, x11, x10   # x5 = 1 if b < a (i.e., a > b)
```

- x5 now stores **condition** (1 = true, 0 = false)

---

### Step 2: Branch to Else if Condition False

```
beq x5, x0, ELSE   # if x5 == 0, jump to ELSE
```

- `beq` = branch if equal
- If condition is false, jump to ELSE block

---

### Step 3: If Block (True)

```
mv x12, x10        # c = a
j END_IF            # jump past else block
```

---

### Step 4: Else Block

```
ELSE:
mv x12, x11        # c = b
```

---

### Step 5: End of If-Else

```
END_IF:      # Continue with program
```

# Complete RISC-V Assembly:

```
slt  x5, x11, x10   # x5 = 1 if a > b
beq  x5, x0, ELSE   # if not, jump to ELSE
mv  x12, x10        # c = a
j  END_IF           # jump to end
ELSE:
mv  x12, x11        # c = b
END_IF:
# next instructions
```

**Code Example 6.16** IF/ELSE STATEMENT

| High-Level Code | RISC-V Assembly Code |
|---|---|
| `if (apples == oranges)`<br>`  f = g + h;`<br>`else`<br>`  apples = oranges - h;` | `# s0 = apples, s1 = oranges`<br>`# s2 = f, s3 = g, s4 = h`<br>`  bne s0, s1, L1 # skip if (apples != oranges)`<br>`  add s2, s3, s4 # f = g + h`<br>`  j  L2`<br>`L1: sub s0, s1, s4 # apples = oranges - h`<br>`L2:` |

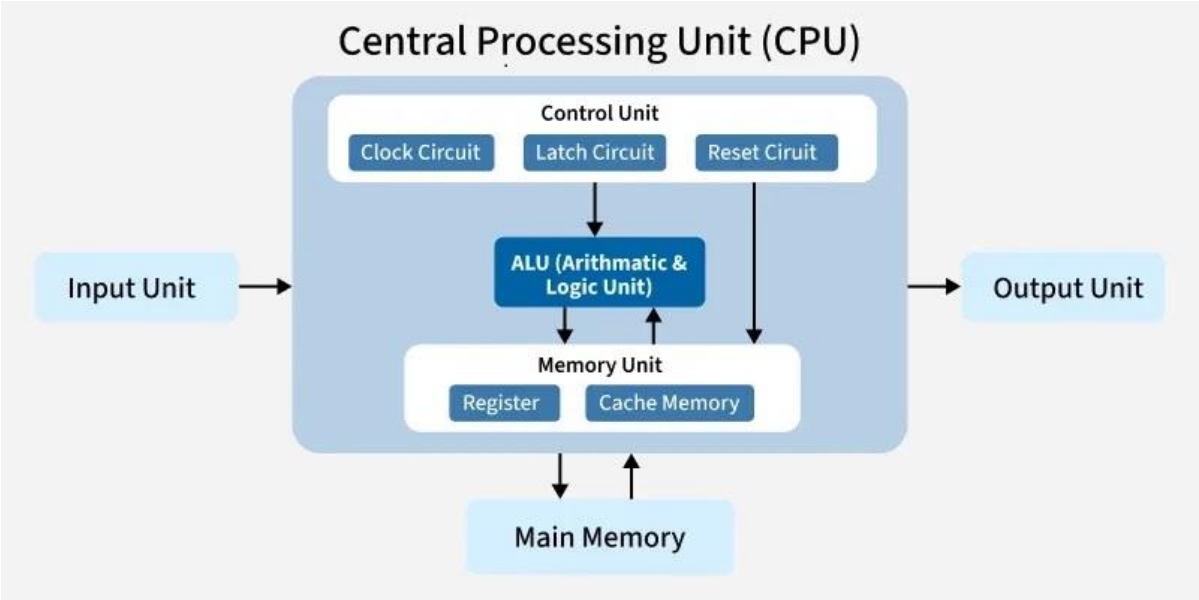
.....

## Q10. CPU Internal Architecture

**A**, Draw and explain the internal architecture of a basic CPU.
**B**, Illustrate the Dataflow from Memory to ALU during execution and explain the role of the different registers involved.

## Ans:

# A,




### 1. Program Counter (PC)

- Stores the **address of the current instruction**
- Automatically increments to next instruction

---

### 2. Instruction Memory

- Stores **program instructions**

- Instruction at address in PC is fetched

---

### 3. Instruction Register (IR)

- Holds the **currently fetched instruction**
- Instruction is decoded from here

### 4. Control Unit

- Decodes the instruction
- Generates **control signals** for ALU, memory, registers

---

### 5. Register File

- Stores operands and intermediate results
- Has **read ports and write ports**

---

### 6. Arithmetic Logic Unit (ALU)

- Performs arithmetic and logical operations
- Also calculates **effective address** for load/store

---

### 7. Data Memory

- Stores data values
- Used by `lw` and `sw` instructions

---

### 8. Multiplexers (MUX)

- Select correct inputs for ALU and registers
- Controlled by control signals

**B,** Dataflow from Memory to ALU

1. Instruction and data are fetched from Main Memory.
2. Data is loaded into Registers.
3. Registers send data to the ALU.
4. ALU performs the operation.
5. Result is stored back in Registers / Memory.

.....

## Q11. Single Cycle Datapath

Demonstrate a Single Cycle Datapath diagram including all necessary control signals.

## Ans,

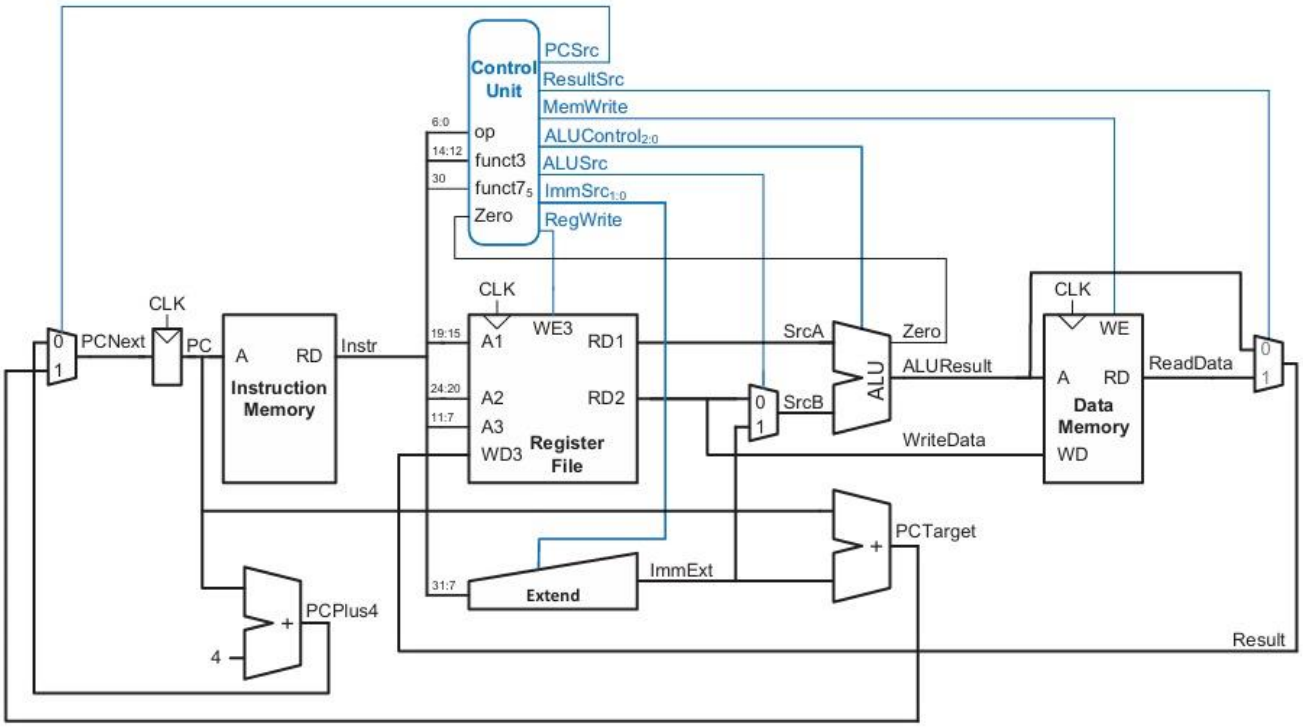



**Figure 7.12** Complete single-cycle processor

## Control Signals (Single-Cycle Processor):

In a single-cycle processor, the **Control Unit** generates control signals based on the instruction to control the datapath in one clock cycle.

## Inputs to Control Unit

- **Opcode (op)** – identifies the instruction type
- **funct3** – specifies the operation
- **funct7** – distinguishes similar operations

## Main Control Signals

- **RegWrite** – writes data into the register file
- **ImmSrc** – selects immediate value format
- **ALUSrc** – selects ALU second input
- **ALUControl** – selects ALU operation
- **MemWrite** – enables writing to data memory
- **ResultSrc** – selects data to write back
- **PCSrc** – selects next PC value